EthereumPriceForecasting

November 7, 2025

Enhanced Ethereum Price Analysis Project

1. Imports and Spark Session

[1]:
```
from pyspark.sql import SparkSession, Window
import pyspark.sql.functions as F
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression, RandomForestRegressor,␣
↪GBTRegressor
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go

spark = SparkSession.builder.appName("EthereumPriceForecastingInteractive").
↪getOrCreate()
```

2. Data Loading

[ ]:
```
df = spark.read.csv("ethereum_dataset.csv", header=True, inferSchema=True)
df = df.dropna(subset=["Open", "High", "Low", "Close"])
```

3. Add Gaussian Noise

[3]:
```
def add_noise(value, sd=0.3):
    if value is None:
        return None
    return float(value) + np.random.normal(0, sd)

noise_udf = F.udf(add_noise, DoubleType())
for col in ['Open', 'High', 'Low', 'Close']:
    df = df.withColumn(col + '_noisy', noise_udf(df[col]))
```

4. Data Cleaning & Validation

[4]:
```
df = df.dropDuplicates(['Unix Timestamp', 'Symbol'])
```

```
for col in ['Open', 'High', 'Low', 'Close', 'Volume']:
    mean_val = df.agg({col: 'mean'}).first()[0]
    df = df.na.fill({col: mean_val})

for col in ['Open', 'High', 'Low', 'Close']:
    mean = df.agg({col: "mean"}).first()[0]
    stddev = df.agg({col: "stddev"}).first()[0]
    z = (F.col(col) - mean) / stddev
    df = df.withColumn(col + '_outlier', F.when(F.abs(z) > 3, 1).otherwise(0))
```

5. Feature Engineering

[5]:
```
w = Window.orderBy("Unix Timestamp").rowsBetween(-6, 0)
w_lag = Window.orderBy("Unix Timestamp")

df = df.withColumn("ma7_close", F.avg("Close").over(w))
df = df.withColumn("lag_close", F.lag("Close", 1).over(w_lag))
df = df.withColumn("log_return", F.log(F.col("Close") / F.col("lag_close")))
df = df.withColumn("volatility_7", F.stddev("Close").over(w))
df = df.withColumn("momentum", (F.col("Close") - F.col("lag_close")) / F.
 ↪col("lag_close"))
df = df.withColumn("price_range", F.col("High") - F.col("Low"))
df = df.na.fill(0)
```

6. Data Preparation for ML

[6]:
```
features = ['ma7_close', 'log_return', 'volatility_7', 'momentum',␣
 ↪'price_range', 'Volume']
assembler = VectorAssembler(inputCols=features, outputCol="features")
model_df = assembler.transform(df).dropna(subset=["features", "Close"])
```

7. Regression Models

[7]:
```
lr = LinearRegression(featuresCol="features", labelCol="Close")
rf = RandomForestRegressor(featuresCol="features", labelCol="Close",␣
 ↪numTrees=100, maxDepth=8)
gbt = GBTRegressor(featuresCol="features", labelCol="Close", maxIter=50)

models = {"Linear Regression": lr, "Random Forest": rf, "Gradient Boosted␣
 ↪Trees": gbt}
predictions = {}
for name, model in models.items():
    fitted = model.fit(model_df)
    preds = fitted.transform(model_df)
    predictions[name] = (fitted, preds)
```

8. Model Evaluation

[10]:
```
evaluator = RegressionEvaluator(labelCol="Close", predictionCol="prediction",␣
 ↪metricName="rmse")

def evaluate(preds, name):
    rmse = evaluator.evaluate(preds)
    mae = RegressionEvaluator(labelCol="Close", predictionCol="prediction",␣
 ↪metricName="mae").evaluate(preds)
    r2 = RegressionEvaluator(labelCol="Close", predictionCol="prediction",␣
 ↪metricName="r2").evaluate(preds)
    return {"Model": name, "RMSE": rmse, "MAE": mae, "R2": r2}

results = [evaluate(preds, name) for name, (_, preds) in predictions.items()]
results_df = pd.DataFrame(results).sort_values("RMSE")
best_model_name = results_df.iloc[0]['Model']
print(results_df)
```

```
                   Model       RMSE        MAE        R2
0      Linear Regression   0.853847   0.297877  0.999988
2  Gradient Boosted Trees  22.418897   9.646211  0.991383
1           Random Forest  37.660026  22.395016  0.975685
```

9. Hyperparameter Tuning (Linear Regression Example)

[11]:
```
paramGrid = (ParamGridBuilder()
             .addGrid(lr.regParam, [0.01, 0.1, 1.0])
             .addGrid(lr.elasticNetParam, [0.0, 0.5])
             .build())

cv = CrossValidator(estimator=lr, estimatorParamMaps=paramGrid,␣
 ↪evaluator=evaluator, numFolds=5)
cv_model = cv.fit(model_df)
best_lr = cv_model.bestModel
print(f"Best Tuned RMSE: {evaluator.evaluate(best_lr.transform(model_df)):.3f}")
```

```
Best Tuned RMSE: 0.854
```

10. Interactive Visualizations

(a) Actual vs Predicted Chart

[12]:
```
best_model, best_pred = predictions[best_model_name]
pdf = best_pred.select("Unix Timestamp", "Date", "Close", "prediction").
 ↪toPandas()
pdf = pdf.sort_values("Date")

fig = go.Figure()
fig.add_trace(go.Scatter(x=pdf["Date"], y=pdf["Close"], mode='lines',␣
 ↪name='Actual Close', line=dict(color='blue')))
```

```
fig.add_trace(go.Scatter(x=pdf["Date"], y=pdf["prediction"], mode='lines',␣
 ↪name='Predicted Close', line=dict(color='orange')))
fig.update_layout(title=f'  Ethereum Price Forecast - {best_model_name}',
                  xaxis_title='Date', yaxis_title='Price (USD)',
                  hovermode='x unified', template='plotly_dark')
fig.show()
```

(b) RMSE Comparison Bar Chart

[13]:
```
fig2 = px.bar(results_df, x="Model", y="RMSE", color="Model",
              title="Model Performance Comparison (Lower RMSE = Better)",
              hover_data=["MAE", "R2"], text_auto=True)
fig2.update_layout(template='plotly_dark', xaxis_title="Model",␣
 ↪yaxis_title="RMSE")
fig2.show()
```

(c) Feature Importance

[14]:
```
rf_model, _ = predictions["Random Forest"]
importances = rf_model.featureImportances.toArray()
fi_df = pd.DataFrame({"Feature": features, "Importance": importances}).
 ↪sort_values("Importance", ascending=False)

fig3 = px.bar(fi_df, x="Importance", y="Feature", orientation='h',
              title="  Feature Importance (Random Forest)", text_auto=True,
              color="Importance", color_continuous_scale="viridis")
fig3.update_layout(template='plotly_dark')
fig3.show()
```

(d) Residual Distribution

[15]:
```
pdf["residuals"] = pdf["Close"] - pdf["prediction"]
fig4 = px.histogram(pdf, x="residuals", nbins=50, title="Residual Distribution␣
 ↪(Actual - Predicted)",
                    color_discrete_sequence=["#00cc96"])
fig4.update_layout(template='plotly_dark', bargap=0.1)
fig4.show()
```

(e) Correlation Heatmap

[16]:
```
corr = pdf[["Close", "prediction"]].corr()
fig5 = px.imshow(corr, text_auto=True, color_continuous_scale="RdBu_r",
                 title="Correlation Between Actual and Predicted Prices")
fig5.update_layout(template='plotly_dark')
fig5.show()
```

11. Gratitude Note

[17]:
```
print("\n  I sincerely thank my guide/sir for their valuable guidance,␣
  ↪encouragement, and insights that made this project successful.")
```

```
  I sincerely thank my guide/sir for their valuable guidance, encouragement, and
insights that made this project successful.
```